JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class OffsetDateTime

java.lang.Object
    java.time.OffsetDateTime

All Implemented Interfaces:

Serializable, Comparable<OffsetDateTime>, Temporal, TemporalAccessor, TemporalAdjuster

---

```
public final class OffsetDateTime
extends Object
implements Temporal, TemporalAdjuster, Comparable<OffsetDateTime>, Serializable
```

A date-time with an offset from UTC/Greenwich in the ISO-8601 calendar system, such as `2007-12-03T10:15:30+01:00`.

`OffsetDateTime` is an immutable representation of a date-time with an offset. This class stores all date and time fields, to a precision of nanoseconds, as well as the offset from UTC/Greenwich. For example, the value "2nd October 2007 at 13:45:30.123456789 +02:00" can be stored in an `OffsetDateTime`.

`OffsetDateTime`, `ZonedDateTime` and `Instant` all store an instant on the time-line to nanosecond precision. `Instant` is the simplest, simply representing the instant. `OffsetDateTime` adds to the instant the offset from UTC/Greenwich, which allows the local date-time to be obtained. `ZonedDateTime` adds full time-zone rules.

It is intended that `ZonedDateTime` or `Instant` is used to model data in simpler applications. This class may be used when modeling date-time concepts in more detail, or when communicating to a database or in a network protocol.

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail. The `equals` method should be used for comparisons.

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
static final OffsetDateTime	MAX	The maximum supported OffsetDateTime, '+999999999-12-31T23:59:59.999999999-18:00'.
static final OffsetDateTime	MIN	The minimum supported OffsetDateTime, '-999999999-01-01T00:00:00+18:00'.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
Temporal	adjustInto(Temporal temporal)	Adjusts the specified temporal object to have the same offset, date and time as this object.
ZonedDateTime	atZoneSameInstant(ZoneId zone)	Combines this date-time with a time-zone to create a ZonedDateTime ensuring that the result has the same instant.
ZonedDateTime	atZoneSimilarLocal(ZoneId zone)	Combines this date-time with a time-zone to create a ZonedDateTime trying to keep the same local date and time.
int	compareTo(OffsetDateTime other)	Compares this date-time to another date-time.
boolean	equals(Object obj)	Checks if this date-time is equal to another date-time.

String	**format** (DateTimeFormatter forma	Formats this date-time using the specified formatter.
static OffsetDateTime	**from** (TemporalAccessor tempor	Obtains an instance of OffsetDateTime from a temporal object.
int	**get** (TemporalField field)	Gets the value of the specified field from this date-time as an int.
int	**getDayOfMonth**()	Gets the day-of-month field.
DayOfWeek	**getDayOfWeek**()	Gets the day-of-week field, which is an enum DayOfWeek.
int	**getDayOfYear**()	Gets the day-of-year field.
int	**getHour**()	Gets the hour-of-day field.
long	**getLong** (TemporalField field)	Gets the value of the specified field from this date-time as a long.
int	**getMinute**()	Gets the minute-of-hour field.
Month	**getMonth**()	Gets the month-of-year field using the Month enum.
int	**getMonthValue**()	Gets the month-of-year field from 1 to 12.
int	**getNano**()	Gets the nano-of-second field.
ZoneOffset	**getOffset**()	Gets the zone offset, such as '+01:00'.
int	**getSecond**()	Gets the second-of-minute field.

int	**getYear**()	Gets the year field.
int	**hashCode**()	A hash code for this date-time.
boolean	**isAfter**(OffsetDateTime other)	Checks if the instant of this date-time is after that of the specified date-time.
boolean	**isBefore**(OffsetDateTime other)	Checks if the instant of this date-time is before that of the specified date-time.
boolean	**isEqual**(OffsetDateTime other)	Checks if the instant of this date-time is equal to that of the specified date-time.
boolean	**isSupported**(TemporalField field)	Checks if the specified field is supported.
boolean	**isSupported**(TemporalUnit unit)	Checks if the specified unit is supported.
OffsetDateTime	**minus**(long amountToSubtract, TemporalUnit unit)	Returns a copy of this date-time with the specified amount subtracted.
OffsetDateTime	**minus**(TemporalAmount amountTo	Returns a copy of this date-time with the specified amount subtracted.
OffsetDateTime	**minusDays**(long days)	Returns a copy of this `OffsetDateTime` with the specified number of days subtracted.
OffsetDateTime	**minusHours**(long hours)	Returns a copy of this `OffsetDateTime` with the specified number of hours subtracted.
OffsetDateTime		

	minusMinutes(long minutes)	Returns a copy of this OffsetDateTime with the specified number of minutes subtracted.
OffsetDateTime	**minusMonths**(long months)	Returns a copy of this OffsetDateTime with the specified number of months subtracted.
OffsetDateTime	**minusNanos**(long nanos)	Returns a copy of this OffsetDateTime with the specified number of nanoseconds subtracted.
OffsetDateTime	**minusSeconds**(long seconds)	Returns a copy of this OffsetDateTime with the specified number of seconds subtracted.
OffsetDateTime	**minusWeeks**(long weeks)	Returns a copy of this OffsetDateTime with the specified number of weeks subtracted.
OffsetDateTime	**minusYears**(long years)	Returns a copy of this OffsetDateTime with the specified number of years subtracted.
static OffsetDateTime	**now**()	Obtains the current date-time from the system clock in the default time-zone.
static OffsetDateTime	**now**(Clock clock)	Obtains the current date-time from the specified clock.
static OffsetDateTime	**now**(ZoneId zone)	Obtains the current date-time from the system clock in the specified time-zone.
static OffsetDateTime	**of**(int year, int month, int dayOfMonth,	Obtains an instance of OffsetDateTime from a

	int hour, int minute, int second, int nanoOfSecond, ZoneOffset offset)	year, month, day, hour, minute, second, nanosecond and offset.
static OffsetDateTime	**of**(LocalDate date, LocalTime time, ZoneOffset offset)	Obtains an instance of OffsetDateTime from a date, time and offset.
static OffsetDateTime	**of**(LocalDateTime dateTime, ZoneOffset offset)	Obtains an instance of OffsetDateTime from a date-time and offset.
static OffsetDateTime	**ofInstant**(Instant instant, ZoneId zone)	Obtains an instance of OffsetDateTime from an Instant and zone ID.
static OffsetDateTime	**parse**(CharSequence text)	Obtains an instance of OffsetDateTime from a text string such as 2007-12-03T10:15:30+01:00.
static OffsetDateTime	**parse**(CharSequence text, DateTimeFormatter format	Obtains an instance of OffsetDateTime from a text string using a specific formatter.
OffsetDateTime	**plus**(long amountToAdd, TemporalUnit unit)	Returns a copy of this date-time with the specified amount added.
OffsetDateTime	**plus**(TemporalAmount amountTo	Returns a copy of this date-time with the specified amount added.
OffsetDateTime	**plusDays**(long days)	Returns a copy of this OffsetDateTime with the specified number of days added.
OffsetDateTime	**plusHours**(long hours)	Returns a copy of this OffsetDateTime with the specified number of hours added.

OffsetDateTime	**plusMinutes**(long minutes)	Returns a copy of this OffsetDateTime with the specified number of minutes added.
OffsetDateTime	**plusMonths**(long months)	Returns a copy of this OffsetDateTime with the specified number of months added.
OffsetDateTime	**plusNanos**(long nanos)	Returns a copy of this OffsetDateTime with the specified number of nanoseconds added.
OffsetDateTime	**plusSeconds**(long seconds)	Returns a copy of this OffsetDateTime with the specified number of seconds added.
OffsetDateTime	**plusWeeks**(long weeks)	Returns a copy of this OffsetDateTime with the specified number of weeks added.
OffsetDateTime	**plusYears**(long years)	Returns a copy of this OffsetDateTime with the specified number of years added.
<R> R	**query**(TemporalQuery<R> query)	Queries this date-time using the specified query.
ValueRange	**range**(TemporalField field)	Gets the range of valid values for the specified field.
static Comparator<OffsetDateTime>	**timeLineOrder**()	Gets a comparator that compares two OffsetDateTime instances based solely on the instant.
long	**toEpochSecond**()	Converts this date-time to the number of

		seconds from the epoch of 1970-01-01T00:00:00Z.
Instant	**toInstant**()	Converts this date-time to an Instant.
LocalDate	**toLocalDate**()	Gets the LocalDate part of this date-time.
LocalDateTime	**toLocalDateTime**()	Gets the LocalDateTime part of this date-time.
LocalTime	**toLocalTime**()	Gets the LocalTime part of this date-time.
OffsetTime	**toOffsetTime**()	Converts this date-time to an OffsetTime.
String	**toString**()	Outputs this date-time as a String, such as 2007-12-03T10:15:30+01:00.
ZonedDateTime	**toZonedDateTime**()	Converts this date-time to a ZonedDateTime using the offset as the zone ID.
OffsetDateTime	**truncatedTo**(TemporalUnit unit)	Returns a copy of this OffsetDateTime with the time truncated.
long	**until**(Temporal endExclusive, TemporalUnit unit)	Calculates the amount of time until another date-time in terms of the specified unit.
OffsetDateTime	**with**(TemporalAdjuster adjust	Returns an adjusted copy of this date-time.
OffsetDateTime	**with**(TemporalField field, long newValue)	Returns a copy of this date-time with the specified field set to a new value.

OffsetDateTime	**withDayOfMonth**(int dayOfMonth)	Returns a copy of this OffsetDateTime with the day-of-month altered.
OffsetDateTime	**withDayOfYear**(int dayOfYear)	Returns a copy of this OffsetDateTime with the day-of-year altered.
OffsetDateTime	**withHour**(int hour)	Returns a copy of this OffsetDateTime with the hour-of-day altered.
OffsetDateTime	**withMinute**(int minute)	Returns a copy of this OffsetDateTime with the minute-of-hour altered.
OffsetDateTime	**withMonth**(int month)	Returns a copy of this OffsetDateTime with the month-of-year altered.
OffsetDateTime	**withNano**(int nanoOfSecond)	Returns a copy of this OffsetDateTime with the nano-of-second altered.
OffsetDateTime	**withOffsetSameInstant**(ZoneOffset offset)	Returns a copy of this OffsetDateTime with the specified offset ensuring that the result is at the same instant.
OffsetDateTime	**withOffsetSameLocal**(ZoneOffset offset)	Returns a copy of this OffsetDateTime with the specified offset ensuring that the result has the same local date-time.
OffsetDateTime	**withSecond**(int second)	Returns a copy of this OffsetDateTime with the second-of-minute altered.
OffsetDateTime	**withYear**(int year)	Returns a copy of this OffsetDateTime with the year altered.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Field Details

MIN

public static final OffsetDateTime MIN

The minimum supported OffsetDateTime, '-999999999-01-01T00:00:00+18:00'. This is the local date-time of midnight at the start of the minimum date in the maximum offset (larger offsets are earlier on the time-line). This combines LocalDateTime.MIN and ZoneOffset.MAX. This could be used by an application as a "far past" date-time.

MAX

public static final OffsetDateTime MAX

The maximum supported OffsetDateTime, '+999999999-12-31T23:59:59.999999999-18:00'. This is the local date-time just before midnight at the end of the maximum date in the minimum offset (larger negative offsets are later on the time-line). This combines LocalDateTime.MAX and ZoneOffset.MIN. This could be used by an application as a "far future" date-time.

Method Details

timeLineOrder

public static Comparator<OffsetDateTime> timeLineOrder()

Gets a comparator that compares two OffsetDateTime instances based solely on the instant.

This method differs from the comparison in compareTo(OffsetDateTime) in that it only compares the underlying instant.

Returns:

a comparator that compares in time-line order

See Also:

isAfter(OffsetDateTime), isBefore(OffsetDateTime), isEqual(OffsetDateTime)

now

```
public static OffsetDateTime now()
```

Obtains the current date-time from the system clock in the default time-zone.

This will query the `system clock` in the default time-zone to obtain the current date-time. The offset will be calculated from the time-zone in the clock.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Returns:

the current date-time using the system clock, not null

now

```
public static OffsetDateTime now(ZoneId zone)
```

Obtains the current date-time from the system clock in the specified time-zone.

This will query the `system clock` to obtain the current date-time. Specifying the time-zone avoids dependence on the default time-zone. The offset will be calculated from the specified time-zone.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Parameters:

`zone` - the zone ID to use, not null

Returns:

the current date-time using the system clock, not null

now

```
public static OffsetDateTime now(Clock clock)
```

Obtains the current date-time from the specified clock.

This will query the specified clock to obtain the current date-time. The offset will be calculated from the time-zone in the clock.

Using this method allows the use of an alternate clock for testing. The alternate clock may be introduced using `dependency injection`.

Parameters:

`clock` - the clock to use, not null

Returns:

the current date-time, not null

of

```
public static OffsetDateTime of(LocalDate date,
                                LocalTime time,
                                ZoneOffset offset)
```

Obtains an instance of `OffsetDateTime` from a date, time and offset.

This creates an offset date-time with the specified local date, time and offset.

Parameters:

`date` - the local date, not null

`time` - the local time, not null

`offset` - the zone offset, not null

Returns:

the offset date-time, not null

of

```
public static OffsetDateTime of(LocalDateTime dateTime,
                                ZoneOffset offset)
```

Obtains an instance of `OffsetDateTime` from a date-time and offset.

This creates an offset date-time with the specified local date-time and offset.

Parameters:

`dateTime` - the local date-time, not null

`offset` - the zone offset, not null

Returns:

the offset date-time, not null

of

```
public static OffsetDateTime of(int year,
                                 int month,
                                 int dayOfMonth,
                                 int hour,
                                 int minute,
                                 int second,
                                 int nanoOfSecond,
                                 ZoneOffset offset)
```

Obtains an instance of `OffsetDateTime` from a year, month, day, hour, minute, second, nanosecond and offset.

This creates an offset date-time with the seven specified fields.

This method exists primarily for writing test cases. Non test-code will typically use other methods to create an offset time. `LocalDateTime` has five additional convenience variants of the equivalent factory method taking fewer arguments. They are not provided here to reduce the footprint of the API.

Parameters:

`year` - the year to represent, from MIN_YEAR to MAX_YEAR

`month` - the month-of-year to represent, from 1 (January) to 12 (December)

`dayOfMonth` - the day-of-month to represent, from 1 to 31

`hour` - the hour-of-day to represent, from 0 to 23

`minute` - the minute-of-hour to represent, from 0 to 59

`second` - the second-of-minute to represent, from 0 to 59

`nanoOfSecond` - the nano-of-second to represent, from 0 to 999,999,999

`offset` - the zone offset, not null

Returns:

the offset date-time, not null

Throws:

`DateTimeException` - if the value of any field is out of range, or if the day-of-month is invalid for the month-year

ofInstant

```
public static OffsetDateTime ofInstant(Instant instant,
                                       ZoneId zone)
```

Obtains an instance of `OffsetDateTime` from an `Instant` and zone ID.

This creates an offset date-time with the same instant as that specified. Finding the offset from UTC/Greenwich is simple as there is only one valid offset for each instant.

Parameters:

`instant` - the instant to create the date-time from, not null

`zone` - the time-zone, which may be an offset, not null

Returns:

the offset date-time, not null

Throws:

`DateTimeException` - if the result exceeds the supported range

from

```
public static OffsetDateTime from(TemporalAccessor temporal)
```

Obtains an instance of `OffsetDateTime` from a temporal object.

This obtains an offset date-time based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `OffsetDateTime`.

The conversion will first obtain a `ZoneOffset` from the temporal object. It will then try to obtain a `LocalDateTime`, falling back to an `Instant` if necessary. The result will be the combination of `ZoneOffset` with either with `LocalDateTime` or `Instant`. Implementations are permitted to perform optimizations such as accessing those fields that are equivalent to the relevant objects.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `OffsetDateTime::from`.

Parameters:

`temporal` - the temporal object to convert, not null

Returns:

the offset date-time, not null

Throws:

`DateTimeException` - if unable to convert to an `OffsetDateTime`

parse

```
public static OffsetDateTime parse(CharSequence text)
```

Obtains an instance of `OffsetDateTime` from a text string such as `2007-12-03T10:15:30+01:00`.

The string must represent a valid date-time and is parsed using `DateTimeFormatter.ISO_OFFSET_DATE_TIME`.

Parameters:

`text` - the text to parse such as "2007-12-03T10:15:30+01:00", not null

Returns:

the parsed offset date-time, not null

Throws:

`DateTimeParseException` - if the text cannot be parsed

parse

```
public static OffsetDateTime parse(CharSequence text,
                                   DateTimeFormatter formatter)
```

Obtains an instance of `OffsetDateTime` from a text string using a specific formatter.

The text is parsed using the formatter, returning a date-time.

Parameters:

`text` - the text to parse, not null

`formatter` - the formatter to use, not null

Returns:

the parsed offset date-time, not null

Throws:

`DateTimeParseException` - if the text cannot be parsed

isSupported

```
public boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if this date-time can be queried for the specified field. If false, then calling the `range`, `get` and `with(TemporalField, long)` methods will throw an exception.

If the field is a `ChronoField` then the query is implemented here. The supported fields are:

- `NANO_OF_SECOND`
- `NANO_OF_DAY`

- MICRO_OF_SECOND
- MICRO_OF_DAY
- MILLI_OF_SECOND
- MILLI_OF_DAY
- SECOND_OF_MINUTE
- SECOND_OF_DAY
- MINUTE_OF_HOUR
- MINUTE_OF_DAY
- HOUR_OF_AMPM
- CLOCK_HOUR_OF_AMPM
- HOUR_OF_DAY
- CLOCK_HOUR_OF_DAY
- AMPM_OF_DAY
- DAY_OF_WEEK
- ALIGNED_DAY_OF_WEEK_IN_MONTH
- ALIGNED_DAY_OF_WEEK_IN_YEAR
- DAY_OF_MONTH
- DAY_OF_YEAR
- EPOCH_DAY
- ALIGNED_WEEK_OF_MONTH
- ALIGNED_WEEK_OF_YEAR
- MONTH_OF_YEAR
- PROLEPTIC_MONTH
- YEAR_OF_ERA
- YEAR
- ERA
- INSTANT_SECONDS
- OFFSET_SECONDS

All other `ChronoField` instances will return false.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.isSupportedBy(TemporalAccessor)` passing `this` as the argument. Whether the field is supported is determined by the field.

Specified by:

`isSupported` in interface `TemporalAccessor`

Parameters:

`field` - the field to check, null returns false

Returns:

true if the field is supported on this date-time, false if not

isSupported

```
public boolean isSupported(TemporalUnit unit)
```

Checks if the specified unit is supported.

This checks if the specified unit can be added to, or subtracted from, this date-time. If false, then calling the `plus(long, TemporalUnit)` and `minus` methods will throw an exception.

If the unit is a `ChronoUnit` then the query is implemented here. The supported units are:

- `NANOS`
- `MICROS`
- `MILLIS`
- `SECONDS`
- `MINUTES`
- `HOURS`
- `HALF_DAYS`
- `DAYS`
- `WEEKS`
- `MONTHS`
- `YEARS`
- `DECADES`
- `CENTURIES`
- `MILLENNIA`
- `ERAS`

All other `ChronoUnit` instances will return false.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.isSupportedBy(Temporal)` passing `this` as the argument. Whether the unit is supported is determined by the unit.

Specified by:

`isSupported` in interface `Temporal`

Parameters:

`unit` - the unit to check, null returns false

Returns:

true if the unit can be added/subtracted, false if not

range

```
public ValueRange range(TemporalField field)
```

Gets the range of valid values for the specified field.

The range object expresses the minimum and maximum valid values for a field. This date-time is used to enhance the accuracy of the returned range. If

it is not possible to return the range, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return appropriate range instances. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.rangeRefinedBy(TemporalAccessor)` passing `this` as the argument. Whether the range can be obtained is determined by the field.

Specified by:

`range` in interface `TemporalAccessor`

Parameters:

`field` - the field to query the range for, not null

Returns:

the range of valid values for the field, not null

Throws:

`DateTimeException` - if the range for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

get

```
public int get(TemporalField field)
```

Gets the value of the specified field from this date-time as an `int`.

This queries this date-time for the value of the specified field. The returned value will always be within the valid range of values for the field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return valid values based on this date-time, except `NANO_OF_DAY`, `MICRO_OF_DAY`, `EPOCH_DAY`, `PROLEPTIC_MONTH` and `INSTANT_SECONDS` which are too large to fit in an `int` and throw an `UnsupportedTemporalTypeException`. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.getFrom(TemporalAccessor)` passing `this` as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`get` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field

Throws:

`DateTimeException` - if a value for the field cannot be obtained or the value is outside the range of valid values for the field

`UnsupportedTemporalTypeException` - if the field is not supported or the range of values exceeds an `int`

`ArithmeticException` - if numeric overflow occurs

getLong

```
public long getLong(TemporalField field)
```

Gets the value of the specified field from this date-time as a `long`.

This queries this date-time for the value of the specified field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return valid values based on this date-time. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.getFrom(TemporalAccessor)` passing `this` as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`getLong` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field

Throws:

`DateTimeException` - if a value for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

getOffset

`public ZoneOffset getOffset()`

Gets the zone offset, such as '+01:00'.

This is the offset of the local date-time from UTC/Greenwich.

Returns:

the zone offset, not null

withOffsetSameLocal

`public OffsetDateTime withOffsetSameLocal(ZoneOffset offset)`

Returns a copy of this `OffsetDateTime` with the specified offset ensuring that the result has the same local date-time.

This method returns an object with the same `LocalDateTime` and the specified `ZoneOffset`. No calculation is needed or performed. For example, if this time represents `2007-12-03T10:30+02:00` and the offset specified is `+03:00`, then this method will return `2007-12-03T10:30+03:00`.

To take into account the difference between the offsets, and adjust the time fields, use `withOffsetSameInstant(ZoneOffset)`.

This instance is immutable and unaffected by this method call.

Parameters:

`offset` - the zone offset to change to, not null

Returns:

an `OffsetDateTime` based on this date-time with the requested offset, not null

withOffsetSameInstant

`public OffsetDateTime withOffsetSameInstant(ZoneOffset offset)`

Returns a copy of this `OffsetDateTime` with the specified offset ensuring that the result is at the same instant.

This method returns an object with the specified `ZoneOffset` and a `LocalDateTime` adjusted by the difference between the two offsets. This will result in the old and new objects representing the same instant. This is useful for finding the local time in a different offset. For example, if this time represents `2007-12-03T10:30+02:00` and the offset specified is `+03:00`, then this method will return `2007-12-03T11:30+03:00`.

To change the offset without adjusting the local time use `withOffsetSameLocal(ZoneOffset)`.

This instance is immutable and unaffected by this method call.

Parameters:

`offset` - the zone offset to change to, not null

Returns:

an `OffsetDateTime` based on this date-time with the requested offset, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

toLocalDateTime

```
public LocalDateTime toLocalDateTime()
```

Gets the `LocalDateTime` part of this date-time.

This returns a `LocalDateTime` with the same year, month, day and time as this date-time.

Returns:

the local date-time part of this date-time, not null

toLocalDate

```
public LocalDate toLocalDate()
```

Gets the `LocalDate` part of this date-time.

This returns a `LocalDate` with the same year, month and day as this date-time.

Returns:

the date part of this date-time, not null

getYear

```
public int getYear()
```

Gets the year field.

This method returns the primitive `int` value for the year.

The year returned by this method is proleptic as per `get(YEAR)`. To obtain the year-of-era, use `get(YEAR_OF_ERA)`.

Returns:

the year, from MIN_YEAR to MAX_YEAR

getMonthValue

`public int getMonthValue()`

Gets the month-of-year field from 1 to 12.

This method returns the month as an `int` from 1 to 12. Application code is frequently clearer if the enum `Month` is used by calling `getMonth()`.

Returns:

the month-of-year, from 1 to 12

See Also:

`getMonth()`

getMonth

`public Month getMonth()`

Gets the month-of-year field using the `Month` enum.

This method returns the enum `Month` for the month. This avoids confusion as to what `int` values mean. If you need access to the primitive `int` value then the enum provides the `int value`.

Returns:

the month-of-year, not null

See Also:

`getMonthValue()`

getDayOfMonth

`public int getDayOfMonth()`

Gets the day-of-month field.

This method returns the primitive `int` value for the day-of-month.

Returns:

the day-of-month, from 1 to 31

getDayOfYear

`public int getDayOfYear()`

Gets the day-of-year field.

This method returns the primitive `int` value for the day-of-year.

Returns:

the day-of-year, from 1 to 365, or 366 in a leap year

getDayOfWeek

```
public DayOfWeek getDayOfWeek()
```

Gets the day-of-week field, which is an enum `DayOfWeek`.

This method returns the enum `DayOfWeek` for the day-of-week. This avoids confusion as to what `int` values mean. If you need access to the primitive `int` value then the enum provides the `int value`.

Additional information can be obtained from the `DayOfWeek`. This includes textual names of the values.

Returns:

the day-of-week, not null

toLocalTime

```
public LocalTime toLocalTime()
```

Gets the `LocalTime` part of this date-time.

This returns a `LocalTime` with the same hour, minute, second and nanosecond as this date-time.

Returns:

the time part of this date-time, not null

getHour

```
public int getHour()
```

Gets the hour-of-day field.

Returns:

the hour-of-day, from 0 to 23

getMinute

```
public int getMinute()
```

Gets the minute-of-hour field.

Returns:

the minute-of-hour, from 0 to 59

getSecond

`public int getSecond()`

Gets the second-of-minute field.

Returns:

the second-of-minute, from 0 to 59

getNano

`public int getNano()`

Gets the nano-of-second field.

Returns:

the nano-of-second, from 0 to 999,999,999

with

`public OffsetDateTime with(TemporalAdjuster adjuster)`

Returns an adjusted copy of this date-time.

This returns an `OffsetDateTime`, based on this one, with the date-time adjusted. The adjustment takes place using the specified adjuster strategy object. Read the documentation of the adjuster to understand what adjustment will be made.

A simple adjuster might simply set the one of the fields, such as the year field. A more complex adjuster might set the date to the last day of the month. A selection of common adjustments is provided in `TemporalAdjusters`. These include finding the "last day of the month" and "next Wednesday". Key date-time classes also implement the `TemporalAdjuster` interface, such as `Month` and `MonthDay`. The adjuster is responsible for handling special cases, such as the varying lengths of month and leap years.

For example this code returns a date on the last day of July:

```
import static java.time.Month.*;
import static java.time.temporal.TemporalAdjusters.*;

result = offsetDateTime.with(JULY).with(lastDayOfMonth());
```

The classes `LocalDate`, `LocalTime` and `ZoneOffset` implement `TemporalAdjuster`, thus this method can be used to change the date, time or offset:

```
result = offsetDateTime.with(date);
result = offsetDateTime.with(time);
result = offsetDateTime.with(offset);
```

The result of this method is obtained by invoking the `TemporalAdjuster.adjustInto(Temporal)` method on the specified adjuster passing `this` as the argument.

This instance is immutable and unaffected by this method call.

Specified by:

`with` in interface `Temporal`

Parameters:

`adjuster` - the adjuster to use, not null

Returns:

an `OffsetDateTime` based on `this` with the adjustment made, not null

Throws:

`DateTimeException` - if the adjustment cannot be made

`ArithmeticException` - if numeric overflow occurs

with

```
public OffsetDateTime with(TemporalField field,
                           long newValue)
```

Returns a copy of this date-time with the specified field set to a new value.

This returns an `OffsetDateTime`, based on this one, with the value for the specified field changed. This can be used to change any supported field, such as the year, month or day-of-month. If it is not possible to set the value, because the field is not supported or for some other reason, an exception is thrown.

In some cases, changing the specified field can cause the resulting date-time to become invalid, such as changing the month from 31st January to February would make the day-of-month invalid. In cases like this, the field is responsible for resolving the date. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

If the field is a `ChronoField` then the adjustment is implemented here.

The `INSTANT_SECONDS` field will return a date-time with the specified instant. The offset and nano-of-second are unchanged. If the new instant value is outside the valid range then a `DateTimeException` will be thrown.

The `OFFSET_SECONDS` field will return a date-time with the specified offset. The local date-time is unaltered. If the new offset value is outside the valid range then a `DateTimeException` will be thrown.

The other `supported fields` will behave as per the matching method on `LocalDateTime`. In this case, the offset is not part of the calculation and will be unchanged.

All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.adjustInto(Temporal, long)` passing `this` as the argument. In this case, the field determines whether and how to adjust the instant.

This instance is immutable and unaffected by this method call.

Specified by:

`with` in interface `Temporal`

Parameters:

`field` - the field to set in the result, not null

`newValue` - the new value of the field in the result

Returns:

an `OffsetDateTime` based on `this` with the specified field set, not null

Throws:

`DateTimeException` - if the field cannot be set

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

withYear

`public OffsetDateTime withYear(int year)`

Returns a copy of this `OffsetDateTime` with the year altered.

The time and offset do not affect the calculation and will be the same in the result. If the day-of-month is invalid for the year, it will be changed to the last valid day of the month.

This instance is immutable and unaffected by this method call.

Parameters:

`year` - the year to set in the result, from MIN_YEAR to MAX_YEAR

Returns:

an `OffsetDateTime` based on this date-time with the requested year, not null

Throws:

`DateTimeException` - if the year value is invalid

withMonth

`public OffsetDateTime withMonth(int month)`

Returns a copy of this `OffsetDateTime` with the month-of-year altered.

The time and offset do not affect the calculation and will be the same in the result. If the day-of-month is invalid for the year, it will be changed to the last valid day of the month.

This instance is immutable and unaffected by this method call.

Parameters:

`month` - the month-of-year to set in the result, from 1 (January) to 12 (December)

Returns:

an `OffsetDateTime` based on this date-time with the requested month, not null

Throws:

`DateTimeException` - if the month-of-year value is invalid

withDayOfMonth

`public OffsetDateTime withDayOfMonth(int dayOfMonth)`

Returns a copy of this `OffsetDateTime` with the day-of-month altered.

If the resulting `OffsetDateTime` is invalid, an exception is thrown. The time and offset do not affect the calculation and will be the same in the result.

This instance is immutable and unaffected by this method call.

Parameters:

`dayOfMonth` - the day-of-month to set in the result, from 1 to 28-31

Returns:

an `OffsetDateTime` based on this date-time with the requested day, not null

Throws:

`DateTimeException` - if the day-of-month value is invalid, or if the day-of-month is invalid for the month-year

withDayOfYear

`public OffsetDateTime withDayOfYear(int dayOfYear)`

Returns a copy of this `OffsetDateTime` with the day-of-year altered.

The time and offset do not affect the calculation and will be the same in the result. If the resulting `OffsetDateTime` is invalid, an exception is thrown.

This instance is immutable and unaffected by this method call.

Parameters:

`dayOfYear` - the day-of-year to set in the result, from 1 to 365-366

Returns:

an `OffsetDateTime` based on this date with the requested day, not null

Throws:

`DateTimeException` - if the day-of-year value is invalid, or if the day-of-year is invalid for the year

withHour

`public OffsetDateTime withHour(int hour)`

Returns a copy of this `OffsetDateTime` with the hour-of-day altered.

The date and offset do not affect the calculation and will be the same in the result.

This instance is immutable and unaffected by this method call.

Parameters:

`hour` - the hour-of-day to set in the result, from 0 to 23

Returns:

an `OffsetDateTime` based on this date-time with the requested hour, not null

Throws:

`DateTimeException` - if the hour value is invalid

withMinute

`public OffsetDateTime withMinute(int minute)`

Returns a copy of this `OffsetDateTime` with the minute-of-hour altered.

The date and offset do not affect the calculation and will be the same in the result.

This instance is immutable and unaffected by this method call.

Parameters:

`minute` - the minute-of-hour to set in the result, from 0 to 59

Returns:

an `OffsetDateTime` based on this date-time with the requested minute, not null

Throws:

`DateTimeException` - if the minute value is invalid

withSecond

`public OffsetDateTime withSecond(int second)`

Returns a copy of this `OffsetDateTime` with the second-of-minute altered.

The date and offset do not affect the calculation and will be the same in the result.

This instance is immutable and unaffected by this method call.

Parameters:

`second` - the second-of-minute to set in the result, from 0 to 59

Returns:

an `OffsetDateTime` based on this date-time with the requested second, not null

Throws:

`DateTimeException` - if the second value is invalid

withNano

`public OffsetDateTime withNano(int nanoOfSecond)`

Returns a copy of this `OffsetDateTime` with the nano-of-second altered.

The date and offset do not affect the calculation and will be the same in the result.

This instance is immutable and unaffected by this method call.

Parameters:

`nanoOfSecond` - the nano-of-second to set in the result, from 0 to 999,999,999

Returns:

an `OffsetDateTime` based on this date-time with the requested nanosecond, not null

Throws:

`DateTimeException` - if the nano value is invalid

truncatedTo

```
public OffsetDateTime truncatedTo(TemporalUnit unit)
```

Returns a copy of this `OffsetDateTime` with the time truncated.

Truncation returns a copy of the original date-time with fields smaller than the specified unit set to zero. For example, truncating with the `minutes` unit will set the second-of-minute and nano-of-second field to zero.

The unit must have a duration that divides into the length of a standard day without remainder. This includes all supplied time units on `ChronoUnit` and `DAYS`. Other units throw an exception.

The offset does not affect the calculation and will be the same in the result.

This instance is immutable and unaffected by this method call.

Parameters:

`unit` - the unit to truncate to, not null

Returns:

an `OffsetDateTime` based on this date-time with the time truncated, not null

Throws:

`DateTimeException` - if unable to truncate

`UnsupportedTemporalTypeException` - if the unit is not supported

plus

```
public OffsetDateTime plus(TemporalAmount amountToAdd)
```

Returns a copy of this date-time with the specified amount added.

This returns an `OffsetDateTime`, based on this one, with the specified amount added. The amount is typically `Period` or `Duration` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.addTo(Temporal)`. The amount implementation is free to implement the addition in any way it wishes, however it typically calls back to `plus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully added.

This instance is immutable and unaffected by this method call.

Specified by:

`plus` in interface `Temporal`

Parameters:

`amountToAdd` - the amount to add, not null

Returns:

an `OffsetDateTime` based on this date-time with the addition made, not null

Throws:

`DateTimeException` - if the addition cannot be made

`ArithmeticException` - if numeric overflow occurs

plus

```
public OffsetDateTime plus(long amountToAdd,
                           TemporalUnit unit)
```

Returns a copy of this date-time with the specified amount added.

This returns an `OffsetDateTime`, based on this one, with the amount in terms of the unit added. If it is not possible to add the amount, because the unit is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoUnit` then the addition is implemented by `LocalDateTime.plus(long, TemporalUnit)`. The offset is not part of the calculation and will be unchanged in the result.

If the field is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.addTo(Temporal, long)` passing `this` as the argument. In this case, the unit determines whether and how to perform the addition.

This instance is immutable and unaffected by this method call.

Specified by:

`plus` in interface `Temporal`

Parameters:

`amountToAdd` - the amount of the unit to add to the result, may be negative

`unit` - the unit of the amount to add, not null

Returns:

an `OffsetDateTime` based on this date-time with the specified amount added, not null

Throws:

`DateTimeException` - if the addition cannot be made

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

plusYears

```
public OffsetDateTime plusYears(long years)
```

Returns a copy of this `OffsetDateTime` with the specified number of years added.

This method adds the specified amount to the years field in three steps:

1. Add the input years to the year field
2. Check if the resulting date would be invalid
3. Adjust the day-of-month to the last valid day if necessary

For example, 2008-02-29 (leap year) plus one year would result in the invalid date 2009-02-29 (standard year). Instead of returning an invalid result, the last valid day of the month, 2009-02-28, is selected instead.

This instance is immutable and unaffected by this method call.

Parameters:

`years` - the years to add, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the years added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusMonths

```
public OffsetDateTime plusMonths(long months)
```

Returns a copy of this `OffsetDateTime` with the specified number of months added.

This method adds the specified amount to the months field in three steps:

1. Add the input months to the month-of-year field
2. Check if the resulting date would be invalid
3. Adjust the day-of-month to the last valid day if necessary

For example, 2007-03-31 plus one month would result in the invalid date 2007-04-31. Instead of returning an invalid result, the last valid day of the month, 2007-04-30, is selected instead.

This instance is immutable and unaffected by this method call.

Parameters:

`months` - the months to add, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the months added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusWeeks

`public OffsetDateTime plusWeeks(long weeks)`

Returns a copy of this OffsetDateTime with the specified number of weeks added.

This method adds the specified amount in weeks to the days field incrementing the month and year fields as necessary to ensure the result remains valid. The result is only invalid if the maximum/minimum year is exceeded.

For example, 2008-12-31 plus one week would result in 2009-01-07.

This instance is immutable and unaffected by this method call.

Parameters:

`weeks` - the weeks to add, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the weeks added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusDays

`public OffsetDateTime plusDays(long days)`

Returns a copy of this OffsetDateTime with the specified number of days added.

This method adds the specified amount to the days field incrementing the month and year fields as necessary to ensure the result remains valid. The result is only invalid if the maximum/minimum year is exceeded.

For example, 2008-12-31 plus one day would result in 2009-01-01.

This instance is immutable and unaffected by this method call.

Parameters:

`days` - the days to add, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the days added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusHours

`public OffsetDateTime plusHours(long hours)`

Returns a copy of this `OffsetDateTime` with the specified number of hours added.

This instance is immutable and unaffected by this method call.

Parameters:

`hours` - the hours to add, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the hours added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusMinutes

`public OffsetDateTime plusMinutes(long minutes)`

Returns a copy of this `OffsetDateTime` with the specified number of minutes added.

This instance is immutable and unaffected by this method call.

Parameters:

`minutes` - the minutes to add, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the minutes added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusSeconds

`public OffsetDateTime plusSeconds(long seconds)`

Returns a copy of this `OffsetDateTime` with the specified number of seconds added.

This instance is immutable and unaffected by this method call.

Parameters:

`seconds` - the seconds to add, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the seconds added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusNanos

`public OffsetDateTime plusNanos(long nanos)`

Returns a copy of this `OffsetDateTime` with the specified number of nanoseconds added.

This instance is immutable and unaffected by this method call.

Parameters:

`nanos` - the nanos to add, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the nanoseconds added, not null

Throws:

`DateTimeException` - if the unit cannot be added to this type

minus

`public OffsetDateTime minus(TemporalAmount amountToSubtract)`

Returns a copy of this date-time with the specified amount subtracted.

This returns an `OffsetDateTime`, based on this one, with the specified amount subtracted. The amount is typically `Period` or `Duration` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.subtractFrom(Temporal)`. The amount implementation is free to implement the subtraction in any way it wishes, however it typically calls back to `minus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully subtracted.

This instance is immutable and unaffected by this method call.

Specified by:

`minus` in interface `Temporal`

Parameters:

`amountToSubtract` - the amount to subtract, not null

Returns:

an `OffsetDateTime` based on this date-time with the subtraction made, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`ArithmeticException` - if numeric overflow occurs

minus

```
public OffsetDateTime minus(long amountToSubtract,
                            TemporalUnit unit)
```

Returns a copy of this date-time with the specified amount subtracted.

This returns an `OffsetDateTime`, based on this one, with the amount in terms of the unit subtracted. If it is not possible to subtract the amount, because the unit is not supported or for some other reason, an exception is thrown.

This method is equivalent to `plus(long, TemporalUnit)` with the amount negated. See that method for a full description of how addition, and thus subtraction, works.

This instance is immutable and unaffected by this method call.

Specified by:

`minus` in interface `Temporal`

Parameters:

`amountToSubtract` - the amount of the unit to subtract from the result, may be negative

`unit` - the unit of the amount to subtract, not null

Returns:

an `OffsetDateTime` based on this date-time with the specified amount subtracted, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

minusYears

`public OffsetDateTime minusYears(long years)`

Returns a copy of this `OffsetDateTime` with the specified number of years subtracted.

This method subtracts the specified amount from the years field in three steps:

1. Subtract the input years from the year field
2. Check if the resulting date would be invalid
3. Adjust the day-of-month to the last valid day if necessary

For example, 2008-02-29 (leap year) minus one year would result in the invalid date 2007-02-29 (standard year). Instead of returning an invalid result, the last valid day of the month, 2007-02-28, is selected instead.

This instance is immutable and unaffected by this method call.

Parameters:

`years` - the years to subtract, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the years subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusMonths

`public OffsetDateTime minusMonths(long months)`

Returns a copy of this `OffsetDateTime` with the specified number of months subtracted.

This method subtracts the specified amount from the months field in three steps:

1. Subtract the input months from the month-of-year field
2. Check if the resulting date would be invalid
3. Adjust the day-of-month to the last valid day if necessary

For example, 2007-03-31 minus one month would result in the invalid date 2007-02-31. Instead of returning an invalid result, the last valid day of the month, 2007-02-28, is selected instead.

This instance is immutable and unaffected by this method call.

Parameters:

`months` - the months to subtract, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the months subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusWeeks

`public OffsetDateTime minusWeeks(long weeks)`

Returns a copy of this `OffsetDateTime` with the specified number of weeks subtracted.

This method subtracts the specified amount in weeks from the days field decrementing the month and year fields as necessary to ensure the result remains valid. The result is only invalid if the maximum/minimum year is exceeded.

For example, 2009-01-07 minus one week would result in 2008-12-31.

This instance is immutable and unaffected by this method call.

Parameters:

`weeks` - the weeks to subtract, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the weeks subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusDays

`public OffsetDateTime minusDays(long days)`

Returns a copy of this `OffsetDateTime` with the specified number of days subtracted.

This method subtracts the specified amount from the days field decrementing the month and year fields as necessary to ensure the result remains valid. The result is only invalid if the maximum/minimum year is exceeded.

For example, 2009-01-01 minus one day would result in 2008-12-31.

This instance is immutable and unaffected by this method call.

Parameters:

`days` - the days to subtract, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the days subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusHours

```
public OffsetDateTime minusHours(long hours)
```

Returns a copy of this `OffsetDateTime` with the specified number of hours subtracted.

This instance is immutable and unaffected by this method call.

Parameters:

`hours` - the hours to subtract, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the hours subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusMinutes

```
public OffsetDateTime minusMinutes(long minutes)
```

Returns a copy of this `OffsetDateTime` with the specified number of minutes subtracted.

This instance is immutable and unaffected by this method call.

Parameters:

`minutes` - the minutes to subtract, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the minutes subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusSeconds

`public OffsetDateTime minusSeconds(long seconds)`

Returns a copy of this `OffsetDateTime` with the specified number of seconds subtracted.

This instance is immutable and unaffected by this method call.

Parameters:

`seconds` - the seconds to subtract, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the seconds subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusNanos

`public OffsetDateTime minusNanos(long nanos)`

Returns a copy of this `OffsetDateTime` with the specified number of nanoseconds subtracted.

This instance is immutable and unaffected by this method call.

Parameters:

`nanos` - the nanos to subtract, may be negative

Returns:

an `OffsetDateTime` based on this date-time with the nanoseconds subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

query

`public <R> R query(TemporalQuery<R> query)`

Queries this date-time using the specified query.

This queries this date-time using the specified query strategy object. The `TemporalQuery` object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

Specified by:

`query` in interface `TemporalAccessor`

Type Parameters:

`R` - the type of the result

Parameters:

`query` - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

`DateTimeException` - if unable to query (defined by the query)

`ArithmeticException` - if numeric overflow occurs (defined by the query)

adjustInto

`public Temporal adjustInto(Temporal temporal)`

Adjusts the specified temporal object to have the same offset, date and time as this object.

This returns a temporal object of the same observable type as the input with the offset, date and time changed to be the same as this.

The adjustment is equivalent to using `Temporal.with(TemporalField, long)` three times, passing `ChronoField.EPOCH_DAY`, `ChronoField.NANO_OF_DAY` and `ChronoField.OFFSET_SECONDS` as the fields.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.with(TemporalAdjuster)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisOffsetDateTime.adjustInto(temporal);
temporal = temporal.with(thisOffsetDateTime);
```

This instance is immutable and unaffected by this method call.

Specified by:

`adjustInto` in interface `TemporalAdjuster`

Parameters:

`temporal` - the target object to be adjusted, not null

Returns:

the adjusted object, not null

Throws:

`DateTimeException` - if unable to make the adjustment

`ArithmeticException` - if numeric overflow occurs

until

```
public long until(Temporal endExclusive,
                  TemporalUnit unit)
```

Calculates the amount of time until another date-time in terms of the specified unit.

This calculates the amount of time between two `OffsetDateTime` objects in terms of a single `TemporalUnit`. The start and end points are `this` and the specified date-time. The result will be negative if the end is before the start. For example, the amount in days between two date-times can be calculated using `startDateTime.until(endDateTime, DAYS)`.

The `Temporal` passed to this method is converted to a `OffsetDateTime` using `from(TemporalAccessor)`. If the offset differs between the two date-times, the specified end date-time is normalized to have the same offset as this date-time.

The calculation returns a whole number, representing the number of complete units between the two date-times. For example, the amount in months between 2012-06-15T00:00Z and 2012-08-14T23:59Z will only be one month as it is one minute short of two months.

There are two equivalent ways of using this method. The first is to invoke this method. The second is to use `TemporalUnit.between(Temporal, Temporal)`:

```
// these two lines are equivalent
amount = start.until(end, MONTHS);
amount = MONTHS.between(start, end);
```

The choice should be made based on which makes the code more readable.

The calculation is implemented in this method for `ChronoUnit`. The units `NANOS`, `MICROS`, `MILLIS`, `SECONDS`, `MINUTES`, `HOURS` and `HALF_DAYS`, `DAYS`, `WEEKS`, `MONTHS`, `YEARS`, `DECADES`, `CENTURIES`, `MILLENNIA` and `ERAS` are supported. Other `ChronoUnit` values will throw an exception.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.between(Temporal, Temporal)` passing `this` as the first argument and the converted input temporal as the second argument.

This instance is immutable and unaffected by this method call.

Specified by:

`until` in interface `Temporal`

Parameters:

`endExclusive` - the end date, exclusive, which is converted to an `OffsetDateTime`, not null

`unit` - the unit to measure the amount in, not null

Returns:

the amount of time between this date-time and the end date-time

Throws:

`DateTimeException` - if the amount cannot be calculated, or the end temporal cannot be converted to an `OffsetDateTime`

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

format

```
public String format(DateTimeFormatter formatter)
```

Formats this date-time using the specified formatter.

This date-time will be passed to the formatter to produce a string.

Parameters:

`formatter` - the formatter to use, not null

Returns:

the formatted date-time string, not null

Throws:

`DateTimeException` - if an error occurs during printing

atZoneSameInstant

```
public ZonedDateTime atZoneSameInstant(ZoneId zone)
```

Combines this date-time with a time-zone to create a `ZonedDateTime` ensuring that the result has the same instant.

This returns a `ZonedDateTime` formed from this date-time and the specified time-zone. This conversion will ignore the visible local date-time and use the underlying instant instead. This avoids any problems with local time-line gaps or overlaps. The result might have different values for fields such as hour, minute an even day.

To attempt to retain the values of the fields, use `atZoneSimilarLocal(ZoneId)`. To use the offset as the zone ID, use `toZonedDateTime()`.

Parameters:

`zone` - the time-zone to use, not null

Returns:

the zoned date-time formed from this date-time, not null

atZoneSimilarLocal

```
public ZonedDateTime atZoneSimilarLocal(ZoneId zone)
```

Combines this date-time with a time-zone to create a `ZonedDateTime` trying to keep the same local date and time.

This returns a `ZonedDateTime` formed from this date-time and the specified time-zone. Where possible, the result will have the same local date-time as this object.

Time-zone rules, such as daylight savings, mean that not every time on the local time-line exists. If the local date-time is in a gap or overlap according to the rules then a resolver is used to determine the resultant local time and offset. This method uses `ZonedDateTime.ofLocal(LocalDateTime, ZoneId, ZoneOffset)` to retain the offset from this instance if possible.

Finer control over gaps and overlaps is available in two ways. If you simply want to use the later offset at overlaps then call `ZonedDateTime.withLaterOffsetAtOverlap()` immediately after this method.

To create a zoned date-time at the same instant irrespective of the local time-line, use `atZoneSameInstant(ZoneId)`. To use the offset as the zone ID, use `toZonedDateTime()`.

Parameters:

`zone` - the time-zone to use, not null

Returns:

the zoned date-time formed from this date and the earliest valid time for the zone, not null

toOffsetTime

`public OffsetTime toOffsetTime()`

Converts this date-time to an `OffsetTime`.

This returns an offset time with the same local time and offset.

Returns:

an OffsetTime representing the time and offset, not null

toZonedDateTime

`public ZonedDateTime toZonedDateTime()`

Converts this date-time to a `ZonedDateTime` using the offset as the zone ID.

This creates the simplest possible `ZonedDateTime` using the offset as the zone ID.

To control the time-zone used, see `atZoneSameInstant(ZoneId)` and `atZoneSimilarLocal(ZoneId)`.

Returns:

a zoned date-time representing the same local date-time and offset, not null

toInstant

`public Instant toInstant()`

Converts this date-time to an `Instant`.

This returns an `Instant` representing the same point on the time-line as this date-time.

Returns:

an `Instant` representing the same instant, not null

toEpochSecond

`public long toEpochSecond()`

Converts this date-time to the number of seconds from the epoch of 1970-01-01T00:00:00Z.

This allows this date-time to be converted to a value of the `epoch-seconds` field. This is primarily intended for low-level conversions rather than general application usage.

Returns:

the number of seconds from the epoch of 1970-01-01T00:00:00Z

compareTo

`public int compareTo(OffsetDateTime other)`

Compares this date-time to another date-time.

The comparison is based on the instant then on the local date-time. It is "consistent with equals", as defined by `Comparable`.

For example, the following is the comparator order:

1. `2008-12-03T10:30+01:00`
2. `2008-12-03T11:00+01:00`
3. `2008-12-03T12:00+02:00`
4. `2008-12-03T11:30+01:00`
5. `2008-12-03T12:00+01:00`
6. `2008-12-03T12:30+01:00`

Values #2 and #3 represent the same instant on the time-line. When two values represent the same instant, the local date-time is compared to distinguish them. This step is needed to make the ordering consistent with `equals()`.

Specified by:

`compareTo` in interface `Comparable`<`OffsetDateTime`>

Parameters:

`other` - the other date-time to compare to, not null

Returns:

the comparator value, that is the comparison with the `other`'s instant, if they are not equal; and if equal to the `other`'s instant, the comparison of the `other`'s local date-time

See Also:

`isBefore(OffsetDateTime)`, `isAfter(OffsetDateTime)`

isAfter

`public boolean isAfter(OffsetDateTime other)`

Checks if the instant of this date-time is after that of the specified date-time.

This method differs from the comparison in `compareTo(OffsetDateTime)` and `equals(Object)` in that it only compares the instant of the date-time. This is equivalent to using `dateTime1.toInstant().isAfter(dateTime2.toInstant());`.

Parameters:

`other` - the other date-time to compare to, not null

Returns:

true if this is after the instant of the specified date-time

isBefore

```
public boolean isBefore(OffsetDateTime other)
```

Checks
if the instant of this date-time is before that of the specified date-time.

This method differs from the comparison in `compareTo(OffsetDateTime)` in that it only compares the instant of the date-time. This is equivalent to using `dateTime1.toInstant().isBefore(dateTime2.toInstant());`.

Parameters:

`other` - the other date-time to compare to, not null

Returns:

true if this is before the instant of the specified date-time

isEqual

```
public boolean isEqual(OffsetDateTime other)
```

Checks if the instant of this date-time is equal to that of the specified date-time.

This method differs from the comparison in `compareTo(OffsetDateTime)` and `equals(Object)` in that it only compares the instant of the date-time. This is equivalent to using `dateTime1.toInstant().equals(dateTime2.toInstant());`.

Parameters:

`other` - the other date-time to compare to, not null

Returns:

true if the instant equals the instant of the specified date-time

equals

```
public boolean equals(Object obj)
```

Checks if this date-time is equal to another date-time.

The comparison is based on the local date-time and the offset. To compare for the same instant on the time-line, use `isEqual(OffsetDateTime)`. Only objects of type `OffsetDateTime` are compared, other types return false.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to check, null returns false

Returns:

true if this is equal to the other date-time

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

A hash code for this date-time.

Overrides:

`hashCode` in class `Object`

Returns:

a suitable hash code

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

toString

```
public String toString()
```

Outputs this date-time as a `String`, such as `2007-12-03T10:15:30+01:00`.

The output will be one of the following formats:

- `uuuu-MM-dd'T'HH:mmXXXXX`
- `uuuu-MM-dd'T'HH:mm:ssXXXXX`
- `uuuu-MM-dd'T'HH:mm:ss.SSSXXXXX`
- `uuuu-MM-dd'T'HH:mm:ss.SSSSSSXXXXX`
- `uuuu-MM-dd'T'HH:mm:ss.SSSSSSSSSXXXXX`

The format used will be the shortest that outputs the full value of the time where the omitted parts are implied to be zero. The output is compatible with ISO 8601 if the seconds in the offset are zero.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this date-time, not null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.